How Language Model Hallucinations Can Snowball

Muru Zhang[♡] **Ofir Press**[♡] **William Merrill**[♠] **Alisa Liu**[♡] **Noah A. Smith**[♡♣]
[♡]Paul G. Allen School of Computer Science and Engineering, University of Washington
[♠]New York University
[♣]Allen Institute for Artificial Intelligence
nanami17@cs.washington.edu

arXiv:2305.13534v1 [cs.CL] 22 May 2023

Abstract

A major risk of using language models in practical applications is their tendency to hallucinate incorrect statements. Hallucinations are often attributed to knowledge gaps in LMs, but we hypothesize that in some cases, when justifying previously generated hallucinations, LMs output false claims that they can separately *recognize* as incorrect. We construct three question-answering datasets where ChatGPT and GPT-4 often state an incorrect answer and offer an explanation with at least one incorrect claim. Crucially, we find that ChatGPT and GPT-4 can identify 67% and 87% of their own mistakes, respectively. We refer to this phenomenon as hallucination snowballing: an LM over-commits to early mistakes, leading to more mistakes that it otherwise would not make.[1]

1 Introduction

Language models are increasingly being deployed to interface with humans in open-ended information-seeking and problem-solving settings. Despite their diverse capabilities and extreme fluency, a major open challenge is that LMs still hallucinate by making up facts or citing sources that do not exist (Maynez et al., 2020; Liu et al., 2023, i.a.), often while sounding extremely plausible.

Hallucination is commonly attributed to knowledge gaps in LMs (Zheng et al., 2023), motivating mitigation strategies through retrieval over knowledge bases (Lewis et al., 2020; Shuster et al., 2021; Peng et al., 2023) But, do LMs *only* hallucinate when they do not "know" a fact? We present a setting where LMs often generate hallucinations that they immediately recognize as wrong when presented in isolation. Specifically, after an LM answers a question incorrectly, it usually justifies that answer by making incorrect assertions that it separately acknowledges as incorrect (Figure 1).

[1]Our data and code are available at: https://github.com/Nanami18/Snowballed_Hallucination

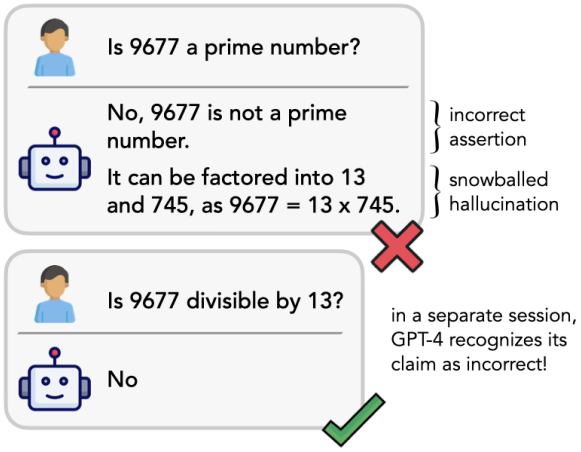

Figure 1: GPT-4 mistakenly claims that 9677 is not prime, followed by an incorrect explanation that $13 \times 745 = 9677$. We refer to this factorization as a *snowballed hallucination*, as GPT-4 appears to "know" that 13 is not a factor of 9677 when asked separately. Thus, hallucinations are not necessarily reflective of a model's lack of "knowledge," but may be induced by an earlier hallucination for consistency.

To study this behavior empirically, we automatically construct three question-answering (QA) datasets. These datasets span different domains: determining whether a number is prime, whether there is a U.S. senator satisfying two given constraints, and whether two cities are connected given a set of flights between cities. Empirically, we find that ChatGPT (OpenAI, 2022) and GPT-4 (OpenAI, 2023) commit to an answer within the first token (`Yes`/`No`) over 95% of the time; these answers are often incorrect, and then followed by an incorrect explanation. Yet, when presented with the incorrect explanation alone, we find that the LM is likely able to recognize it as incorrect.

We refer to this phenomenon as **hallucination snowballing**. We hypothesize that LMs produce snowballed hallucinations for consistency with earlier hallucinations (rather than due to a "knowledge gap" in the model), as they recognize the snow-

Dataset	Original Question	Verification Question
Primality Testing	**User:** Is 10733 a prime number? **GPT-4:** No... It can be factored into 3×3577.	**User:** Is 10733 divisible by 3? Answer with either Yes or No. **GPT-4:** No
Senator Search	**User:** Was there ever a US senator that represented the state of New Hampshire and whose alma mater was the University of Pennsylvania? **GPT-4:** Yes... His name was John P. Hale	**User:** Was John P. Hale's alma mater University of Pennsylvania? **GPT-4:** No... [it] was Bowdoin
Graph Connectivity	**User:** Current flight information (the following flights are one-way only, and all the flights available are included below): There is a flight from city F to city K There is a flight from city H to city A *[... 10 other rules cut for space ...]* Question: Is there a series of flights that goes from city B to city E? **GPT-4:** Yes... the route is as follows: ... City K to City G...	**User:** *[...flight information given in the context...]* Based on the above flight information, is City K to City G a valid flight? **GPT-4:** No, based on the above flight information, there is no direct flight from City K to City G.

Table 1: **Original Question**: An example from each dataset and GPT-4's (incorrect) response, with the snowballed hallucination underlined. **Verification Question**: The corresponding query used to probe whether GPT-4 recognizes its claim is incorrect, with its refutation of the hallucination underlined.

balled hallucination is incorrect when presented in isolation (i.e., in a separate interaction session).

While prompting strategies that encourage the LM to reason before stating an answer improve accuracy on the task, our work points to the broader issue that conditioning on faulty context leads LMs to produce extremely simple mistakes that they wouldn't otherwise make. Indeed, when prompting with "*Let's think step by step*" (Kojima et al., 2023), snowballed hallucinations still occur in 95% of cases where the model fails to answer correctly. We observe that sometimes even when "*Let's think step by step*" does lead to the right answer, it uses invalid reasoning chains.

In this paper, we demonstrate the phenomenon of hallucination snowballing by leveraging recent LMs' tendency to state and justify their answers. Rather than over-committing to its previously generated context, we believe that LMs should acknowledge their initial mistake, and then revise their answer. We have indeed observed GPT-4 doing this in a limited number of cases; amplifying this behavior would be beneficial, as well as developing new methods in which LMs can backtrack.

2 Why do we expect hallucination snowballing?

In this section, we explain why we hypothesize that LMs are susceptible to hallucination snowballing. We predict that snowballing will occur on questions with two key properties:

1. **Initial committal:** The prompt leads the LM to first state an answer (*before* outputting the explanation). This applies to many yes/no questions.
2. **Inherently sequential:** Transformers cannot find the answer within one timestep because of their limited reasoning abilities within one timestep.

We now discuss how these properties may lead to snowballed hallucination.

Initial committal. In English and many other languages, speakers often say the final Yes/No answers to questions before explaining their answer. We therefore hypothesize that LMs and especially instruction-tuned LMs (Wei et al., 2021; Sanh et al., 2021; Ouyang et al., 2022; Wang et al., 2022) will reflect this answer format where the answer comes before the explanation. Indeed, on our datasets (presented in §3.1), we observe that GPT-4 and ChatGPT immediately commit to an answer to the question: the first token is `Yes` or `No` 95.67% and 98.40% of the time for GPT-4 and ChatGPT respectively. In the remaining cases, the model often commits to an answer within the first few tokens of the response (e.g., *"There is no record of a U.S. Senator..."*). Crucially, once the LM generates `Yes` or `No`, that token remains in the context, and coherence would require commitment to that choice through the subsequent justification. Thus, the model pro-

duces an answer to a complex question in a *single* timestep, and it then continues by generating an explanation for that answer, which inevitably will be incorrect.

Inherently sequential. Furthermore, transformers cannot solve inherently sequential reasoning problems like primality testing or graph connectivity within a single timestep,[2] as documented in recent theoretical results (Merrill and Sabharwal, 2023).[3] Our graph connectivity and primality datasets are concrete instantiations of these problems. Because the transformer must use one step to answer a question that requires multiple timesteps to answer correctly, it will necessarily sometimes commit to an incorrect answer. We hypothesize that this leads the LM to hallucinate supporting incorrect facts that it otherwise would not generate.

3 Experiments

We design three QA datasets with the properties described in §2 to probe hallucination snowballing, and evaluate ChatGPT and GPT-4. We first check whether the LM returns the correct answer to the given question, and we show that when the model returns the wrong answer, it frequently provides an incorrect explanation for that wrong answer. We automatically extract the incorrect claim in the explanation and ask the same LM to check whether its claim is correct. See Table 1 for a representative example from each dataset.

3.1 Datasets

We design three QA datasets, each containing 500 yes/no questions that we expect are not answerable by transformers in one timestep. To aid evaluation, the questions are designed so that an incorrect answer would be justified with easily verifiable claims.

[2]Technically, this holds only for inputs above a certain hardness level, i.e., the size of the prime number for primality testing, or the size of the graph for graph connectivity.

[3]Merrill and Sabharwal (2023) show that, with a single generation step, bounded-precision transformers cannot solve any problem outside the complexity class TC^0, which corresponds to a highly parallelizable subclass of both L (log-space) and P (polynomial-time). Graph connectivity is an L-complete problem, which means it cannot be in TC^0 unless $\mathsf{TC}^0 = \mathsf{L}$, i.e., all of L can be parallelized to a surprisingly high degree. Primality testing was shown to be in P (Agrawal et al., 2004) but cannot be in TC^0 unless it is also in L; i.e., any n can be factored with $O(\log\log n)$ bits of overhead. In summary, unless standard complexity-theoretic conjectures are false, graph connectivity and primality testing are outside TC^0 and thus are too inherentially sequential for transformers to solve in a single generation (cf. Merrill and Sabharwal, 2023).

For each dataset, we fix one specific label for all examples, so that if the model chooses the incorrect answer (e.g., that 9677 is not prime), it would produce a specific claim to support it (e.g., an incorrect factorization). This enables us to systematically examine model-written justifications for incorrect answers.

Primality testing For this dataset, we query the primality of 500 randomly chosen primes between 1,000 and 20,000; the correct answer is always `Yes`. When the model answers incorrectly, we expect it to justify its answer with an incorrect factorization.

Senator search This dataset consists of 500 questions of the form "*Was there ever a US senator that represented the state of x and whose alma mater was y?*" where x is a U.S. state and y is a U.S. college. For these questions, the correct answer is always `No`. When the model answers incorrectly, we expect it to falsely claim that a particular senator both represented x and attended y.

To create the dataset we consider all U.S. states and a manually constructed list of twelve popular U.S. colleges (see §A for the full list); for each possible pair, we generate a question following the template, and manually remove pairs where the answer is `Yes`.

Graph connectivity For each of the 500 questions in this dataset, we present 12 flights among 14 cities, and ask if there is a sequence of flights from a particular city to another. The problem always corresponds to the same underlying directed graph structure (see §A.1), where flights are edges and cities are nodes. For each instance in the dataset, we randomly assign letters from the English alphabet to name the nodes. To formulate the query, we sample a source city s and destination city t in different subgraphs, with the additional constraint that s corresponds to a source node, and t a leaf node, so that 1-step heuristics cannot be used to solve the problem.

We formulate the problem as a flight-finding question in natural language so that it sounds more natural: in the prompt, we list the twelve flights ("*There is a flight from city F to city K; there is a flight from city G to city N, ...*"), followed by the question *"Is there a series of flights... from s to t?"*. Note the correct answer is always `No`. When the model answers incorrectly, we expect it to justify its answer with a flight that does not exist.